GroundOne: un decision model specialista per la verifica della groundedness nella retrieval-augmented generation locale

Francesco Pio De Dominicis · Relatore: Prof. Paolo Garza
Laurea Magistrale in Ingegneria Informatica, Politecnico di Torino, A.A. 2025/2026

Riassunto della tesi

Motivazione

La retrieval-augmented generation (RAG) dà ai modelli linguistici accesso a documenti privati, e oggi l'intera pipeline può girare su un dispositivo consumer, a vantaggio della privacy, dell'uso offline e dei costi. Il recupero dei documenti, però, non elimina le allucinazioni: le risposte contengono ancora frasi che il contesto recuperato non supporta o che lo contraddicono. Individuarle è necessario su ogni risposta, sulla stessa macchina dalle risorse limitate che ospita il generatore. La località ha una conseguenza tecnica precisa: su un singolo dispositivo leggere testo costa poco, scriverlo costa molto. Il modello legge l'input in un solo forward pass, limitato dal calcolo, ma scrive un token alla volta, e ogni token è limitato dalla banda di memoria necessaria a leggere tutti i pesi; sul portatile della tesi scrivere un token costa da quattordici a sedici volte più che leggerne uno, e una rassegna dell'hardware locale mostra ovunque la stessa asimmetria. I verificatori attuali sono piccoli encoder, che restituiscono un unico punteggio fisso per coppia documento–frase, oppure giudici generativi, che scrivono i verdetti token per token e violano spesso il formato. Una terza famiglia, i *decision model*, risponde a domande chiuse in un solo forward pass leggendo dai logit la probabilità di ciascuna opzione. Non era ancora stato studiato in modo controllato, su hardware consumer, se un decision model compatto e specializzato in un unico task possa eguagliare giudici ed encoder, da dove derivi il suo vantaggio e se le sue probabilità siano utilizzabili.

GroundOne

GroundOne è un decision model specializzato in un'unica decisione: per ogni frase di una risposta RAG, il documento recuperato la supporta, la contraddice o non la menziona? Ogni frase è seguita da un marker, e i verdetti di tutte le K frasi si leggono dai logit di tre lettere di risposta in corrispondenza dei marker, in un solo forward pass e senza generare testo (Figura 1). Modelli Qwen2.5 da 0.5B e 3B parametri sono sottoposti a fine-tuning con LoRA sulle risposte di RAGTruth annotate manualmente, con una loss pesata per classe e ristretta alle tre lettere. Una temperatura e un bias per classe stimati sul dev set calibrano le probabilità, e una soglia scelta sugli stessi dati le traduce in etichette coerenti con le probabilità mostrate all'utente. Tutto, dalla costruzione del dataset all'addestramento, alla valutazione e alla messa in servizio, gira su un portatile Apple M4 con 16 GB di memoria, con MLX e senza servizi esterni.

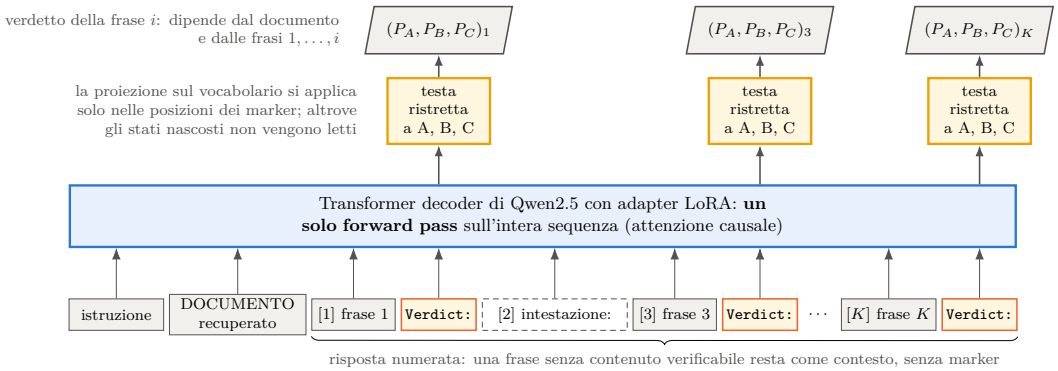


Figura 1: Il prompt a marker di GroundOne: i verdetti di tutte le frasi si leggono in corrispondenza dei marker `Verdict:` in un solo forward pass.

Dati e impostazione sperimentale

Nessuna etichetta è stata annotata ex novo: le annotazioni a livello di span di RAGTruth sono state convertite in un'etichetta a tre classi per frase, con regole fissate in anticipo. Tre test set sono stati costruiti, controllati e congelati prima di qualunque addestramento, con il leakage controllato a livello di documento: la partizione di test di RAGTruth (T1, 17,284 frasi), LLM-AggreFact (T2, undici dataset) e VitaminC (T3, un test di fact checking fuori dominio). GroundOne è confrontato con indizi che non leggono mai il documento, con i modelli non addestrati, con giudici generativi della stessa dimensione, con e senza probabilità, con due decision model generalisti (Rizzo-Flow, 4B, e d1-3B di Liquid AI) e con encoder specializzati (MiniCheck-RoBERTa, HHEM), tutti sulla stessa macchina, con intervalli ottenuti da un bootstrap a cluster.

Risultati

La specializzazione funziona nel dominio per cui è pensata (Tabella 1). Su T1 GroundOne raggiunge una balanced accuracy a tre classi di 0.769 (0.5B) e 0.832 (3B), contro 0.37 e 0.44 prima dell'addestramento; il suo AUROC (0.905 e 0.940) supera quello degli encoder (circa 0.82), e in balanced accuracy a tre classi supera nettamente entrambi i decision model generalisti. Su LLM-AggreFact il 3B è alla pari con MiniCheck-RoBERTa (74.8 contro 74.0, con un intervallo che comprende lo zero). Su VitaminC il guadagno scompare e d1-3B è molto più forte: uno specialista non è uno strumento generale di verifica dei fatti. Il readout conta di per sé: a parità di modello addestrato, scegliere fra le tre lettere ammesse è più accurato che lasciare al modello generare il proprio verdetto (+0.25 per il 0.5B, +0.07 per il 3B), con la stessa latenza. Sulle risposte di quattordici o più frasi GroundOne è da 1.7 a 2 volte più veloce di un giudice sintetico e circa sei volte più veloce di uno verboso, con un output sempre valido, e consuma meno energia (5.5 e 33 joule in media per risposta), anche se il risparmio è minore di quello sul tempo, perché un prefill lungo assorbe più potenza del decoding. Dopo la calibrazione l'Expected Calibration Error del 3B su T1 è 0.019, mentre i giudici a cui si chiede lo stesso output forniscono probabilità che distinguono a stento le decisioni corrette da quelle errate. L'intera valutazione gira senza accesso alla rete con risultati identici bit per bit, i pesi a 8 bit non costano accuratezza rispetto a quelli in bf16, e generatore e verificatore stanno insieme in un unico processo con un picco di 6.1 GB.

Misura	GroundOne 0.5B	GroundOne 3B
Balanced accuracy a tre classi su T1 (non addestrato)	0.769 (0.37)	0.832 (0.44)
AUROC di P(supportata) su T1 (miglior encoder 0.82)	0.905	0.940
Balanced accuracy media su T2 (MiniCheck-RoBERTa 74.0)	70.1	74.8
Readout contro verdetto generato, stessi pesi	+0.25	+0.07
Speed-up con $K \geq 14$ su giudice sintetico / verboso	1.95× / 6.2×	1.73× / 6.2×
Tempo ed energia per risposta	0.41 s, 5.5 J	2.57 s, 33 J

Tabella 1: Risultati principali. I tempi si riferiscono alle risposte di quattordici o più frasi, l'energia è la media per risposta; il 3B non addestrato è stato valutato su 600 risposte di T1.

GroundOne Studio: un'applicazione locale e un caso di studio

Un'applicazione nativa per macOS, scritta in Swift con MLX Swift, mette il verificatore al lavoro in un assistente RAG completo in cui recupero (ricerca full-text di SQLite e un piccolo modello di embedding), generazione (Qwen2.5-3B a 4 bit), verifica e un giudice generativo locale condividono un unico processo, con un picco di memoria di 6.2 GB e senza rete. Un test di conformità su 200 risposte di T1 mostra token identici e probabilità entro 0.005 dall'implementazione Python per il 3B. La risposta del generatore compare per prima; l'utente può poi verificarla con il solo GroundOne o insieme al giudice, consultare le probabilità e il passaggio a supporto di ogni frase e far riscrivere

dalle fonti le frasi segnalate o selezionate, con una richiesta breve per frase che dipende dal tipo di errore. In un caso di studio con 100 errori iniettati GroundOne segnala l'errore in 97 delle 98 risposte in cui la frase riceve un verdetto, il giudice in 84, e la riscrittura mirata riduce dal 100 % al 9 % le risposte che contengono ancora l'errore: numeri e date modificati vengono per lo più ripristinati dalla fonte, le frasi inventate cancellate. Ripetere lo studio con il 0.5B mostra perché il verdetto a tre classi conta: il modello piccolo etichetta i valori modificati come non menzionati anziché contraddetti, e il generatore allora li cancella invece di correggerli. Queste risposte sono accettate da GroundOne, non verificate in modo indipendente.

Limiti e sviluppi futuri

I limiti principali sono la ristrettezza dei dati di addestramento, a causa della quale le contraddizioni al di fuori del data-to-text e le allucinazioni nascoste in frasi per il resto copiate vengono apprese solo in parte, una ricetta di addestramento non del tutto robusta rispetto al seed, un'unica piattaforma e famiglia di modelli e dati soltanto in inglese. Gli sviluppi futuri comprendono una ricetta più robusta (bias correction, gradient clipping, più seed), dati di addestramento con supporto parafrasato e modifiche minime, un porting su llama.cpp per misurare altro hardware, adapter per lingua, l'attribuzione di ogni verdetto a un passaggio, un generatore più grande e un controllo indipendente delle frasi riscritte nell'applicazione.